

minipytorch — Bug Report

Full status of known issues: fixed, actively broken, and still open

Scope: src/cpp, src/pythoninterface, CMakeLists.txt · As of 2026-08-23 · Compiled from a live debugging session plus a full static review

6 fixed & verified	3 active / regressed right now	19 still open, unchanged
-----------------------	-----------------------------------	-----------------------------

Fixed & verified correct

LOCATION	WHAT WAS WRONG	CURRENT STATE
activationnode.cpp Relubbackward	Loop bound used raw byte count instead of element count (4x overrun); wrote into its own gradient buffer instead of the parent's (gradient never propagated); condition checked the incoming gradient's sign instead of the forward input's sign	FIXED All three corrected — loop bound divides by sizeof(float), writes to parents[0]->storageforgrad, condition checks parents[0]->storage->data[i] > 0
factory.cpp empty() / zero() / ones()	Constructor call supplied 4 of 10 required arguments; syntax errors (make_shared<TensorImpl*> missing parens); storageforgrad never allocated	FIXED All three build a full TensorImpl via shared_ptr, storageforgrad allocated and zeroed alongside storage
factory.cpp createbufferforopt()	Referenced undeclared variables (shape, result); unqualified Dtypes/device; no Storage ever allocated; no return statement	FIXED Allocates + zeroes one Storage sized to `size`, returns shared_ptr<Storage>
tensorimpl.hpp tooptmize field	Declared but never set anywhere — every node read garbage/indeterminate when optimizerengine checked it	FIXED Added as 11th constructor parameter, set in constructor body; weight/bias = true, everything else = false
addnode.cpp addbackward	Both loop bounds used raw nbytes (byte count) instead of element count — 4x overrun	FIXED Both loops now divide by sizeof(float) correctly
mulnode.cpp	N/A — was already correct in the original review	CONFIRMED CORRECT resultBytes/resultNumel properly split; unchanged, still correct

Active — needs attention now

These are live bugs in the current file state, including one that got worse mid-fix and one that was reverted back to broken.

LOCATION	CURRENT BUG	IMPACT
addnode.cpp:14-16 add()	ACTIVE resultdatasize is now correctly computed as an <i>element</i> count (nbytes/sizeof(float)) — but it's still passed directly to Storage's constructor at lines 15-16, which expects a <i>byte</i> count. Needs two separate variables (bytes for allocation, elements for simdadd's K param), same split mulnode.cpp already uses (resultBytes / resultNumel).	Under-allocates the output buffer by ~4x. Every call to add() writes past a too-small heap buffer — heap corruption on the most-used op in the graph.
sgdmomentum.cpp:12	REGRESSED An in-progress fix (wrapping state in a struct matching RMSprop's pattern) was reverted. Back to float* velocity = new float[nbytes_as_floats]{}; — fresh, zeroed, on every call.	beta term always multiplies zero — momentum mathematically collapses to plain SGD regardless of the beta value passed in. Also leaks the array every call.

tensorimpl.hpp	NOT YET ADDED The planned shared_ptr<Storage> optimizerState field (agreed design: state lives on the node itself, avoids a dangling-pointer risk that a map-keyed-by-TensorImpl* approach would have) was never added — an earlier attempt at this line was reverted along with the sgdmomentum.cpp regression above.	Blocks the actual fix for sgdmomentum.cpp / sgdnestrov.cpp's state-persistence bugs below.
----------------	---	--

Still open — correctness bugs

LOCATION	BUG	EFFECT
sgdnestrov.cpp:11	OPEN velocity pointer is literally the same pointer as the parameter's own data (thingtoptimizenode->storage->data) instead of independent persistent state.	Velocity gets overwritten by the parameter's live weight value every call — no real accumulated momentum, and the scalar remainder loop (lines 24-26) reads back its own just-overwritten value mid-statement, producing outright wrong numbers even within a single call.
lossfunctionnode.cpp:59-75 BAElOSSfunctionnodebackward	OPEN Meant to be BCE's backward, computes $(1/N)$ $(y_pred - y_true)$ — byte-for-byte the MAE formula copied from directly above it, not the BCE gradient.	Dead code today (never called from lossfunctionnode()), but wrong if ever wired in.
binarycrosslossentropy.cpp:1 maeloss.cpp:1	OPEN #include <iostream.h> — pre-standard header, doesn't exist in any modern compiler.	Neither file compiles as-is.
matrix_benchmark.cpp:27,40,80	OPEN Calls matrixmultiply/transpose/elementwisemultiply, none of which are defined in the file it includes.	The one functioning CMake benchmark target won't link.
matrixsimple.cpp:412-428 generatel	OPEN Indexes into a vector<vector<int>> that was never resized from its default-constructed (zero-row) state.	Undefined behavior on any call to Tensor.identity(size > 0).
matrixsimple.cpp:127-134,171-178,207-214	OPEN Bare throw; used outside any catch block in div_matrix/sqrt_matrix/log_matrix.	Calls std::terminate rather than throwing anything catchable — the surrounding try/catch is decorative for these three paths.

Still open — build & structure

LOCATION	BUG	EFFECT
CMakeLists.txt	OPEN core/, autograd/, and cpu/ (the actual tensor+autograd stack) are referenced by no CMake target at all. Only matrix_ops_simple and matrix_benchmark exist as targets.	The primary subsystem of the project has never been part of the build — only hand-compiled via linear.cpp's main().

linear.cpp:8-10, lossfunctionnode.cpp:5, matrix_benchmark.cpp:4	OPEN .cpp files #included by path instead of compiled+linked separately.	ODR violation risk the moment any of these are also compiled as their own translation unit and linked together.
AutogradEngine.hpp	OPEN build_topo / backward / optimizerengine defined as ordinary (non-inline) functions directly in a header.	Multiple-definition-symbol error the moment a second translation unit includes this header.
Tensor.py:1	OPEN Imports linear_algebra_matrix_operations.matrixcomplex — deleted by a prior commit, only a stale .pyc remains.	The Python-facing Tensor class cannot be imported at all today.
build.bat, .vscode/tasks.json	OPEN Both hardcode a specific developer's local absolute paths (Python install, Intel oneAPI location).	Three divergent, non-portable build procedures exist side by side (CMake+vcpkg, build.bat, tasks.json), none of which build the tensor core.

Still open — ownership & memory

LOCATION	BUG	EFFECT
addnode.cpp, mulnode.cpp, lossfunctionnode.cpp, linear.cpp	OPEN Every TensorImpl is built with raw new and stored as a raw pointer (parents is vector<TensorImpl*>). No delete anywhere in the codebase for this type.	Every forward pass leaks the full graph, including the Storage buffers it holds via shared_ptr internally (kept alive because the owning TensorImpl is never destroyed). Unbounded growth under any training loop.

RMSprop.cpp:17	OPEN (DESIGN NOTE) optimizer_states is keyed by raw TensorImpl*. Safe today only because nothing ever deletes a TensorImpl. The moment the leak above gets fixed, a freed address could be reused by a later `new`, and this map would silently hand the new parameter state state from the old one.	Latent bug that would activate as a direct consequence of fixing the leak, unless the map entry is also cleared when its node is destroyed. (This is the reasoning behind moving optimizer state onto TensorImpl itself instead of a map — see "Active" section above.)
allocator.hpp:20-22 CPUAllocator::free()	OPEN Freed blocks are pushed into a size-keyed pool and reused, but the pool itself never shrinks — no eviction policy.	Long-running processes accumulate unbounded cached memory even though individual buffers are being "freed."
RMSprop.cpp:12-15 sgdmomentum.cpp	OPEN (HYGIENE) Manual new[]/delete[] for per-parameter state instead of std::vector<float>.	Not a correctness bug, but forgoes RAII exactly where per-parameter state needs cleanup alongside its (also leaked) TensorImpl.

Still open — hygiene & consistency

- **dtype.hpp:5** — using namespace std; at global scope in a widely-included header; several autograd node headers similarly leak using namespace mylib::tensor; into every includer.
- **Naming** — optimizers/optimizerengine/optimizerSGD (missing "i") and two different misspellings of "Nesterov" (sgdnestrov, optimizerSGDnestorov) in the same file.
- **Optimizer interfaces** — sgd.h, sgdmomentum.cpp, sgdnestrov.cpp, RMSprop.cpp all have different call signatures with no shared interface; AutogradEngine.hpp's optimizerengine is hardwired to call only optimizerSGD with a hardcoded 0.01 learning rate, so momentum/Nesterov/RMSprop are unreachable from the actual training loop despite existing.
- **.gitignore / repo hygiene** — tracked .pyc files referencing Python source that no longer exists anywhere in the repo; committed build artifacts (test.exe, linear_test.exe, .obj files) despite a .gitignore that mostly excludes them.
- **Header coverage** — only one implementation per optimizer/activation/loss family has a header (sgd.h, relu.h, mseloss.hpp) — the rest are usable only via the .cpp-include-.cpp pattern flagged above.

No test suite

No test source, no framework (gtest/Catch2/doctest), no tests/ directory anywhere in the repository. The closest thing to a test is linear.cpp:69-102's main() — builds a small linear layer, runs forward/backward/optimize once, prints the result, and relies on visually comparing it against a hardcoded expected[] array. No automated assertions.